```python
def lps_length(s):
    n = len(s)
    if n == 0:
        return 0

    dp = [[0] * n for _ in range(n)]

    for i in range(n):
        dp[i][i] = 1

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2:
                    dp[i][j] = 2
                else:
                    dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

    return dp[0][n - 1]


print(lps_length("bbbab"))
print(lps_length("cbbd"))
```

```
4
2

=== Code Execution Successful ==
```

main.py

Visualize | Run

```
3  def lps_length(s):
13     for length in range(2, n + 1):
14         for i in range(n - length + 1):
24
25     return dp[0][n - 1]
26
27
28 print(lps_length("bbbab"))
29 print(lps_length("cbbd"))
30
31 def lcs_length(a, b):
32     n = len(a)
33     m = len(b)
34
35     dp = [[0]*(m+1) for _ in range(n+1)]
36
37     for i in range(1, n+1):
38         for j in range(1, m+1):
39             if a[i-1] == b[j-1]:
40                 dp[i][j] = dp[i-1][j-1] + 1
41             else:
42                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
43
44     return dp[n][m]
45
46 s = "bbbab"
47 print(lcs_length(s, s[::-1]))
```

Output

```
4
2
4

=== Code Execution Successful ===
```

main.py

Visualize | Run

```
def lps_length(s):

    for i in range(n):
        dp[i][i] = 1

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2:
                    dp[i][j] = 2
                else:
                    dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

    return dp[0][n - 1]


print(lps_length("bbbab"))
print(lps_length("cbbd"))

def is_weak_password(password, threshold=0.6):
    return lps_length(password)/len(password) > threshold

print(is_weak_password("bbbab"))
```

Output

```
4
2
True

=== Code Execution Successful ===
```

main.py

Visualize

Run

```python
def lps_length_optimized(s):
    n = len(s)

    dp = [0]*n

    for i in range(n-1, -1, -1):
        prev = 0
        dp[i] = 1

        for j in range(i+1, n):
            temp = dp[j]

            if s[i] == s[j]:
                dp[j] = prev + 2
            else:
                dp[j] = max(dp[j], dp[j-1])

            prev = temp

    return dp[n-1]

print(lps_length_optimized("bbbab"))
```

Output

```
4

=== Code Execution Successful ===
```

main.py

Visualize | Run

```
1  def reconstruct_lps(s):
9      for length in range(2, n+1):
10         for i in range(n-length+1):
18             else:
20
21     i, j = 0, n-1
22     left = ""
23     right = ""
24
25     while i <= j:
26         if i == j:
27             left += s[i]
28             break
29
30         if s[i] == s[j]:
31             left += s[i]
32             right = s[j] + right
33             i += 1
34             j -= 1
35         elif dp[i+1][j] > dp[i][j-1]:
36             i += 1
37         else:
38             j -= 1
39
40     return left + right
41
42 print(reconstruct_lps("bbbab"))
```

Output

```
bbbb

=== Code Execution Successful ===
```

main.py

Visualize | Run

```
print(lps_length("bbbab"))
print(lps_length("cbbd"))

def palindrome_score(word):
    return lps_length(word)

print(palindrome_score("bbbab"))


def symmetry_ratio(msg):
    if len(msg) == 0:
        return 0
    return lps_length(msg) / len(msg)

print(symmetry_ratio("bbbab"))

def base_pair_estimate(s):
    return lps_length(s) // 2

print(base_pair_estimate("bbbab"))


```

Output

```
4
2
4
0.8
2
True

=== Code Execution Successful ===
```

main.py

Visualize

Run

```
3  def lps_length(s):
13     for length in range(2, n + 1):
14         for i in range(n - length + 1):
24
25     return dp[0][n - 1]
26
27
28 print(lps_length("bbbab"))
29 print(lps_length("cbbd"))
30
31 def palindrome_score(word):
32     return lps_length(word)
33
34 print(palindrome_score("bbbab"))
35
36
37 def symmetry_ratio(msg):
38     if len(msg) == 0:
39         return 0
40     return lps_length(msg) / len(msg)
41
42 print(symmetry_ratio("bbbab"))
43
44 def base_pair_estimate(s):
45     return lps_length(s) // 2
46
47 print(base_pair_estimate("bbbab"))
```

Output

```
4
2
4
0.8
2

=== Code Execution Successful ===
```

main.py

Visualize | Run

```
3  def lps_length(s):
13     for length in range(2, n + 1):
14         for i in range(n - length + 1):
17             if s[i] == s[j]:
20                 else:
21                     dp[i][j] = dp[i + 1][j - 1] + 2
22             else:
23                 dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
24
25     return dp[0][n - 1]
26
27
28 print(lps_length("bbbab"))
29 print(lps_length("cbbd"))
30
31 def palindrome_score(word):
32     return lps_length(word)
33
34 print(palindrome_score("bbbab"))
35
36
37 def symmetry_ratio(msg):
38     if len(msg) == 0:
39         return 0
40     return lps_length(msg) / len(msg)
41
42 print(symmetry_ratio("bbbab"))
```

Output

```
4
2
4
0.8

=== Code Execution Successful ===
```

main.py

Visualize | Run

```
3  def lps_length(s):
10     for i in range(n):
11         dp[i][i] = 1
12
13     for length in range(2, n + 1):
14         for i in range(n - length + 1):
15             j = i + length - 1
16
17             if s[i] == s[j]:
18                 if length == 2:
19                     dp[i][j] = 2
20                 else:
21                     dp[i][j] = dp[i + 1][j - 1] + 2
22             else:
23                 dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
24
25     return dp[0][n - 1]
26
27
28 print(lps_length("bbbab"))
29 print(lps_length("cbbd"))
30
31 def palindrome_score(word):
32     return lps_length(word)
33
34 print(palindrome_score("bbbab"))
35
```

Output

```
4
2
4

=== Code Execution Successful ===
```